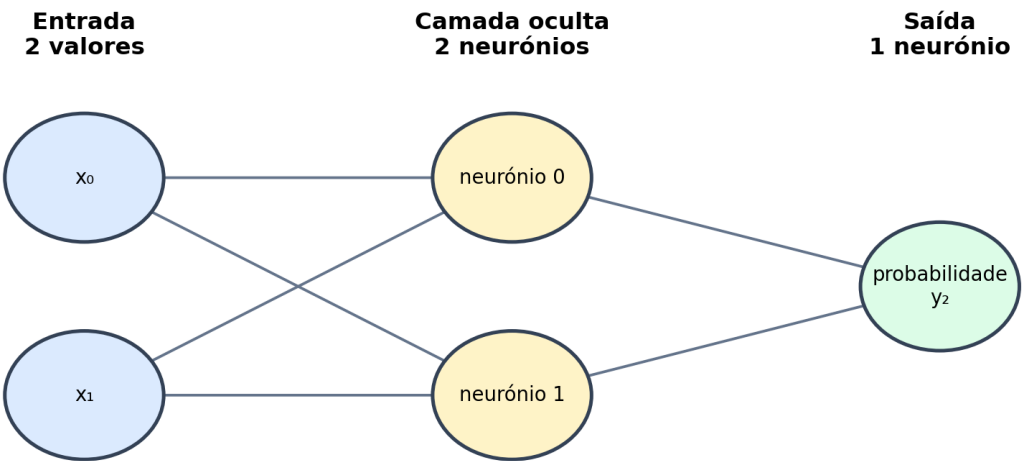


GUIA DO PROJETO DE REDE NEURAL

Leitura do exemplo4.py, aplicações práticas, tarefas do Jira e soluções possíveis

Arquitetura da rede manual do exemplo4.py: 2 → 2 → 1



Cada seta representa um peso: quanto uma informação influencia a seguinte. Os bias deslocam os limites de decisão.

Projeto Jira	NNPROJ — Neural Network Moon Project
Equipa	3 executores + Daniela como reviewer
Data de entrega	26/09/2026
Código de partida	exemplo4.py — aula de 21/08/2026

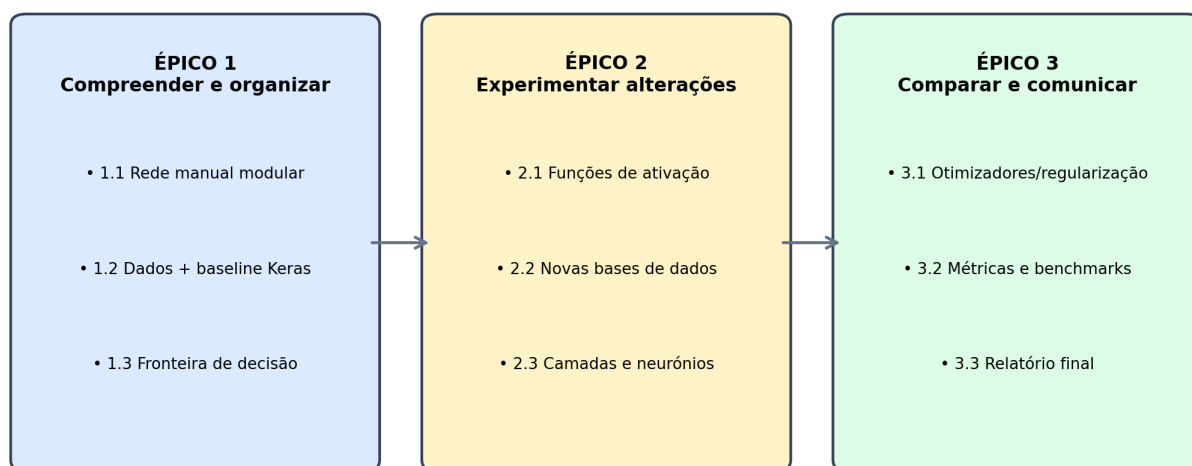
Nota pedagógica: as soluções incluídas são exemplos de caminhos possíveis. O objetivo é que o grupo compreenda, execute, compare e justifique as escolhas. Não existe uma única arquitetura correta.

Índice

- 1. O problema explicado antes da matemática
- 2. Dicionário: tradução da linguagem técnica
- 3. Explicação detalhada do exemplo4.py
- 4. O que o treino está realmente a fazer
- 5. Aplicabilidade prática e real
- 6. Plano das tarefas do Jira
- 7. Possíveis soluções, código e explicação por tarefa
- 8. Como comparar experiências corretamente
- 9. Roteiro da reviewer e checklist de entrega
- 10. Limitações e melhorias do código atual

Este guia começa pela intuição. A matemática e o código aparecem depois, quando já sabemos que pergunta cada linha está a tentar responder.

Mapa do projeto NNPROJ

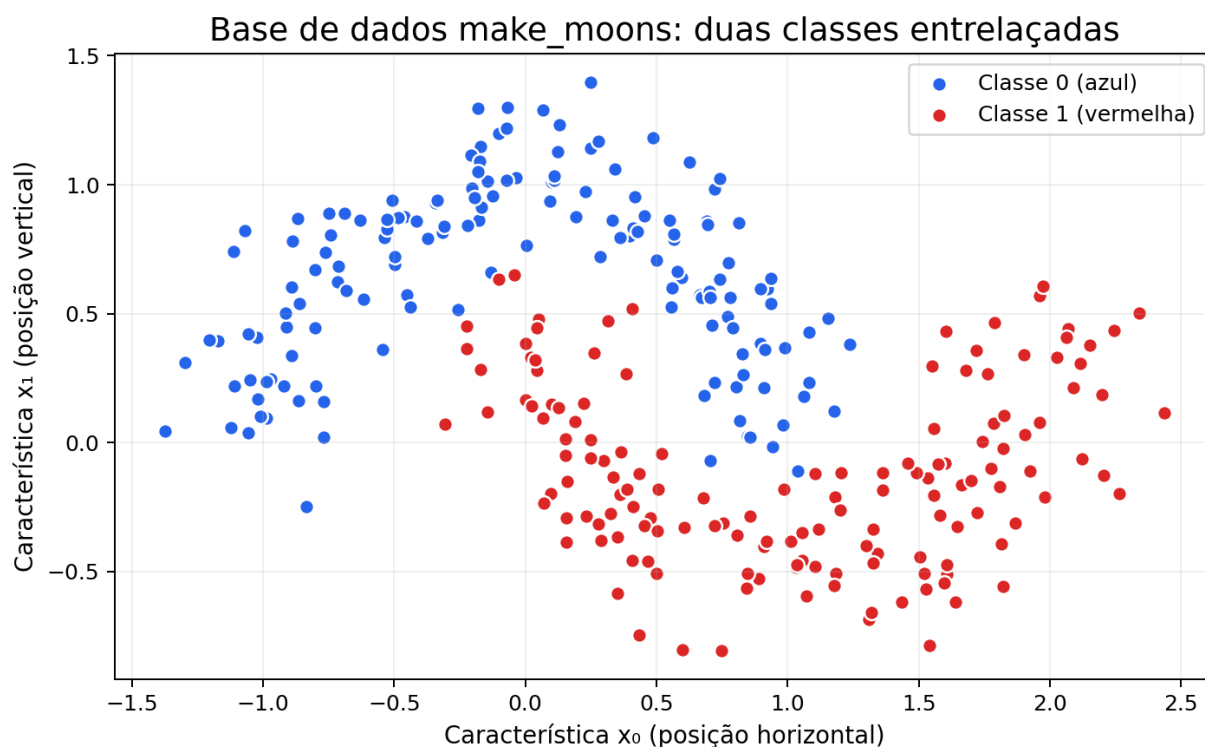


Daniela: reviewer — verifica entendimento, resultados, justificações e qualidade da entrega.

Visão geral das nove tarefas distribuídas pelos três épicos.

1. O problema explicado antes da matemática

O programa recebe pontos com duas coordenadas e tenta decidir a que grupo pertence cada ponto. Os grupos formam duas meias-luas encaixadas. A resposta correta de cada ponto já é conhecida: classe 0 ou classe 1. A rede observa exemplos, erra, corrige os seus parâmetros internos e volta a tentar.



Cada ponto é uma amostra. A cor é a resposta correta que a rede deve aprender.

Uma linha reta não separa bem estas duas formas. Por isso, o exercício é interessante: obriga o modelo a combinar várias pequenas decisões para formar uma fronteira curva. A camada oculta é precisamente o mecanismo que dá essa flexibilidade.

Amostra, característica e classe

Em linguagem simples	Nome técnico
Uma amostra é um ponto. As características são as duas coordenadas usadas como pistas. A classe é a resposta azul (0) ou vermelha (1).	X é a matriz de features com forma $(n_amostras, 2)$. Y é o vetor de labels binárias com forma $(n_amostras,)$.

O que significa aprender?

Em linguagem simples	Nome técnico
Aprender é descobrir valores para os botões internos da rede que façam menos previsões erradas.	Treinar é otimizar pesos e bias minimizando uma função de perda através de gradiente descendente e backpropagation.

2. Dicionário: tradução da linguagem técnica

Termo	Tradução simples
Rede neural	Um conjunto de pequenas unidades de cálculo ligadas entre si.
Neurónio	Uma calculadora: combina entradas, pesos e bias, depois aplica uma função.
Peso (weight)	A importância atribuída a uma informação. Peso alto = grande influência.
Bias ou viés	Um ajuste adicional que desloca o ponto em que o neurónio muda de opinião.
Camada oculta	Etapa intermédia que transforma as pistas para criar decisões não lineares.
Ativação	Regra que transforma o resultado de um neurónio. Ex.: sigmoid, tanh, ReLU.
Forward pass	Caminho para a frente: dos dados até à previsão.
Loss	Número que mede quanto a previsão se afastou da resposta correta.
Gradiente	Indicação da direção em que cada parâmetro aumenta mais o erro.
Backpropagation	Processo de repartir a responsabilidade do erro pelas ligações da rede.
Taxa de aprendizagem	Tamanho do passo usado ao corrigir pesos e bias.
Época	Uma passagem de treino pelo conjunto de exemplos.
Batch	Grupo de exemplos processado antes de atualizar os parâmetros.
Acurácia	Porcentagem de previsões classificadas corretamente.
Overfitting	Quando o modelo memoriza o treino e piora em dados novos.

3. Explicação detalhada do exemplo4.py

3.1 Importações — linhas 1 a 7

```
from sklearn import datasets
import numpy as np
import matplotlib.pyplot as plt
from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import SGD
from keras import initializers
```

sklearn cria a base de dados; NumPy trabalha com vetores e matrizes; Matplotlib cria o gráfico; Keras constrói a segunda rede, de modo automático. A importação initializers não é usada no código atual e pode ser removida sem alterar o resultado.

3.2 Criação e visualização dos dados — linhas 9 a 14

```
X, Y = datasets.make_moons(100, noise=0.1)
color = ['blue' if k == 0 else 'red' for k in Y]
plt.scatter(X[:, 0], X[:, 1], c=color)
plt.savefig('duas_luas.svg')
```

São criados 100 pontos. O parâmetro noise=0.1 acrescenta pequenas irregularidades, tornando o problema mais parecido com dados reais. $X[:, 0]$ significa «todas as linhas da primeira coluna»; $X[:, 1]$ significa «todas as linhas da segunda coluna». Y guarda a resposta correta de cada ponto.

O código não define random_state. Portanto, cada execução pode gerar pontos diferentes e resultados diferentes. Para um trabalho comparável, recomenda-se fixar uma seed.

3.3 A função sigmoid — linhas 17 a 18

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

A sigmoid comprime qualquer número para o intervalo entre 0 e 1. Isso permite tratar a saída como um grau de confiança. Exemplo: 0,92 pode ser interpretado como forte confiança na classe 1. O limite de 0,5 converte esse valor contínuo numa decisão 0 ou 1.

Tecnicamente, a derivada usada depois é $\text{sigmoid}(x) * (1 - \text{sigmoid}(x))$. O código já possui a saída y, por isso escreve simplesmente $y * (1-y)$.

3.4 run_neural_net: apenas prever — linhas 21 a 39

Esta função executa o caminho para a frente. Ela não aprende e não altera pesos. Recebe um ponto e os parâmetros já existentes, calcula os dois neurónios ocultos, calcula o neurónio de saída e devolve a classe prevista.

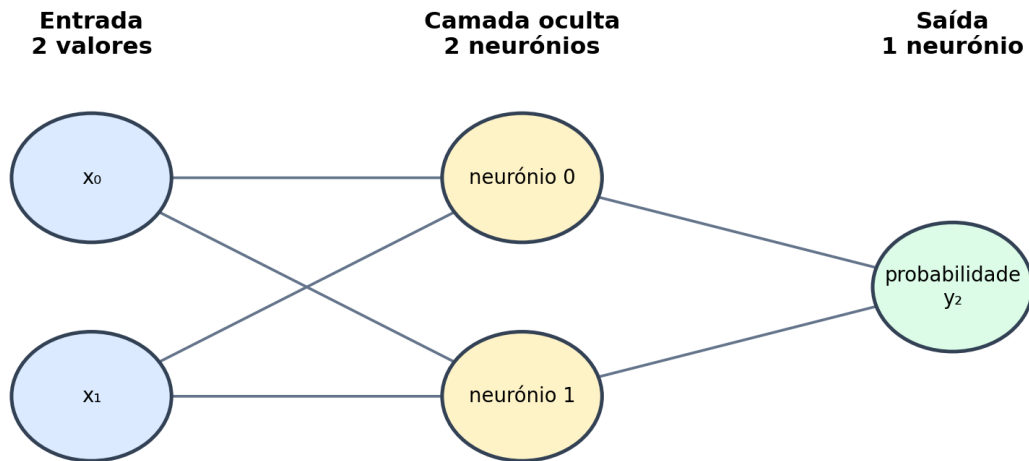
```
s00 = w0[0, 0] * x[0]
s01 = w0[0, 1] * x[1]
s02 = s00 + s01
v0 = s02 + b0[0]
y0 = sigmoid(v0)
```

Tradução linha a linha: multiplicar a primeira coordenada pelo primeiro peso; multiplicar a segunda pelo segundo peso; somar as duas contribuições; acrescentar o bias; passar o resultado pela sigmoid. O segundo neurónio oculto repete a mesma receita com outros pesos.

```
v2 = y0 * w1[0] + y1 * w1[1] + b1[0]
y2 = sigmoid(v2)
return 1 if y2 > 0.5 else 0
```

O neurónio final combina as duas opiniões intermédias. Se a confiança na classe 1 ultrapassar 50%, devolve 1; caso contrário, devolve 0.

Arquitetura da rede manual do exemplo4.py: 2 → 2 → 1



Cada seta representa um peso: quanto uma informação influencia a seguinte. Os bias deslocam os limites de decisão.

3.5 neural_net: prever, medir o erro e calcular correções — linhas 42 a 103

A primeira metade repete o forward pass, mas mantém todos os valores intermédios necessários para calcular o erro e os gradientes.

```
e = y2 - d
L = 1/2 * (e ** 2)
```

d é a resposta desejada. e é a diferença entre a previsão e essa resposta. A loss quadrática torna o erro sempre positivo e penaliza mais os erros grandes. O fator $1/2$ é usado porque simplifica a derivada: a derivada de $1/2 \times e^2$ é e .

Na segunda metade, o código percorre mentalmente a rede no sentido contrário. Começa no erro da saída e calcula quanto cada peso e bias contribuiu para esse erro. Isso é backpropagation.

```
grad_v2 = e * y2 * (1 - y2)
grad_w1[0] = grad_v2 * y0
grad_w1[1] = grad_v2 * y1
```

A primeira linha mede quanto o valor anterior à sigmoid da saída deve mudar. As linhas seguintes medem a responsabilidade dos pesos que ligam os neurónios ocultos à saída. O mesmo raciocínio continua até chegar aos pesos ligados às duas entradas.

3.6 main: inicialização, treino e avaliação — linhas 105 a 154

```
w0 = np.random.rand(2, 2)
w1 = np.random.rand(2)
b0 = np.random.rand(2)
b1 = np.random.rand(1)
taxa = 0.1
```

A rede começa com opiniões aleatórias. A taxa 0,1 controla a força de cada correção. Se for demasiado alta, a rede pode saltar de um lado para o outro; se for demasiado baixa, pode aprender muito lentamente.

```
for i in range(10000):
    ...
    for k in range(100):
        g_w0, g_b0, g_w1, g_b1, L = neural_net(...)
        grad_w0 += g_w0
        ...
    w0 -= taxa * grad_w0
```

Em cada uma das 10.000 épocas, o código calcula os gradientes das 100 amostras e soma-os. Depois atualiza os parâmetros uma vez. Isso é batch gradient descent.

A soma dos gradientes não é dividida por 100. Logo, a dimensão da atualização cresce com o número de amostras. Uma versão mais estável usaria a média dos gradientes ou uma taxa menor.

3.7 A versão Keras — linhas 156 a 165

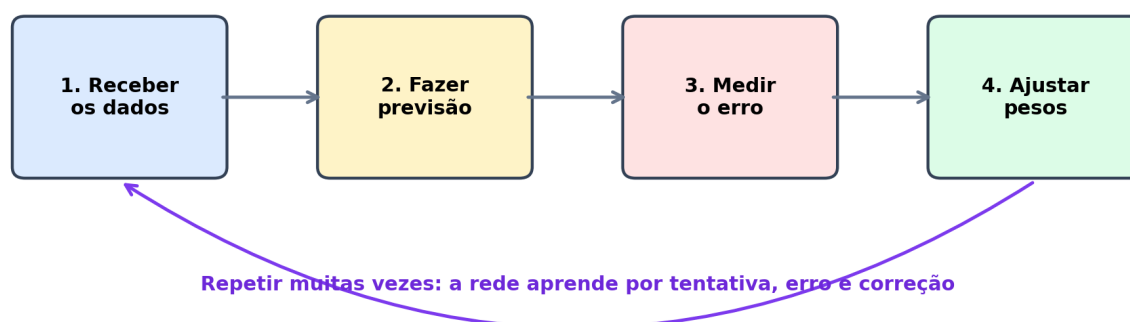
```
model = Sequential()
model.add(Dense(5, input_dim=2, activation='relu'))
model.add(Dense(5, activation='tanh'))
model.add(Dense(1, activation='sigmoid'))
opt = SGD(learning_rate=taxa)
model.compile(loss='mean_squared_error', optimizer=opt, metrics=['accuracy'])
model.fit(X, Y, epochs=100, verbose=False, batch_size=5)
acc = model.evaluate(X, Y)
```

Keras executa automaticamente as multiplicações, as ativações, o backpropagation e as atualizações. Esta rede é $2 \rightarrow 5 \rightarrow 5 \rightarrow 1$, portanto é diferente da rede manual $2 \rightarrow 2 \rightarrow 1$. A comparação atual não isola uma única mudança.

Para classificação binária, `binary_crossentropy` costuma ser uma escolha mais adequada do que erro quadrático. Além disso, o código avalia no mesmo conjunto usado no treino e não imprime o resultado de `model.evaluate`. Para medir generalização, deve separar treino e teste.

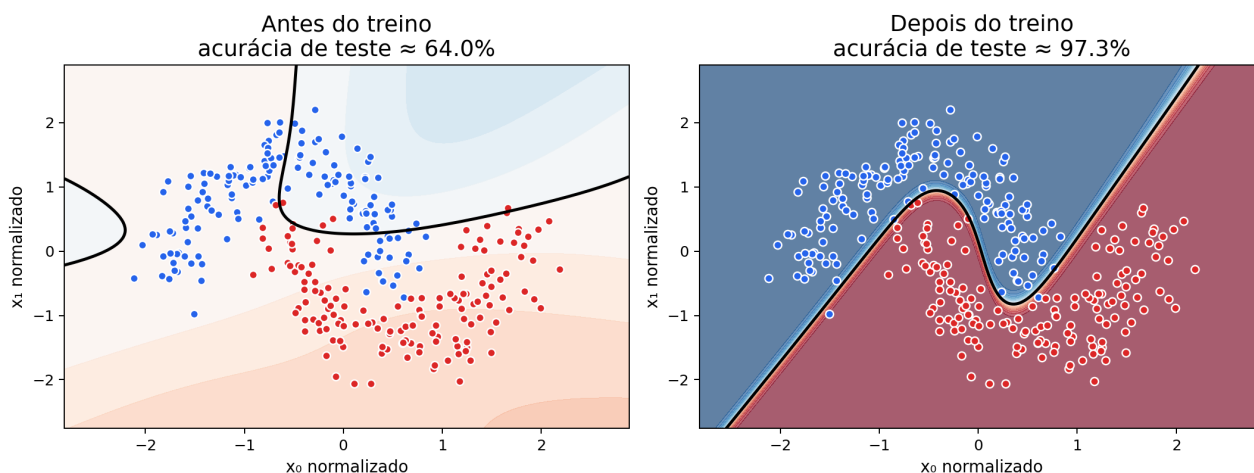
4. O que o treino está realmente a fazer

O ciclo de treino em linguagem simples

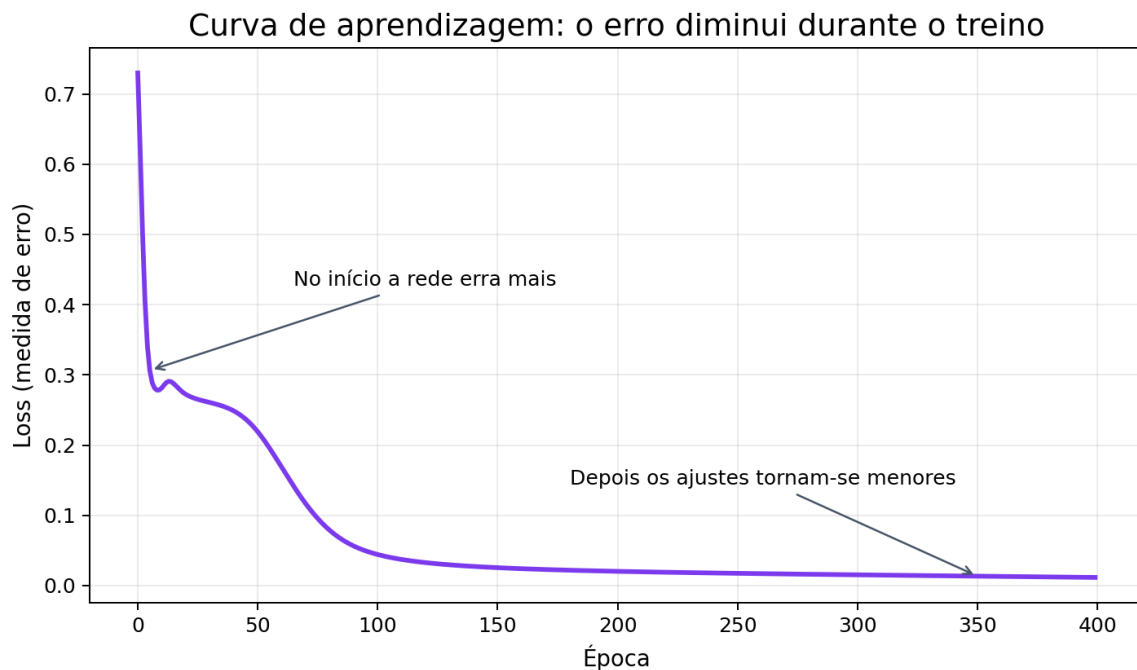


O treino não guarda uma regra escrita por uma pessoa. Ele ajusta números. Depois de muitas correções, esses números combinam-se para produzir uma fronteira que separa as classes. A linha preta abaixo é essa regra aprendida.

A fronteira preta é a regra aprendida pela rede



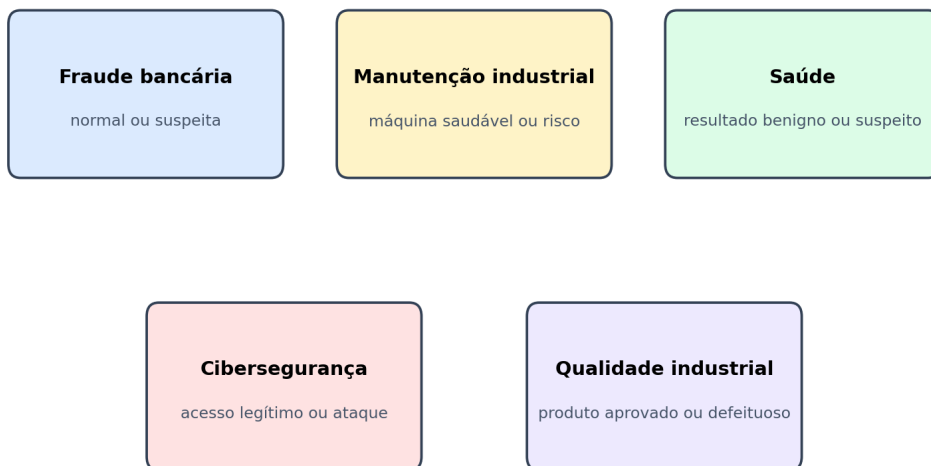
Experiência reproduzível equivalente: a acurácia de teste passou de cerca de 64% para 97,3%.



Uma loss descendente é um bom sinal, mas não basta. Devemos acompanhar também a loss de validação/teste. Se a loss de treino continuar a cair enquanto a de validação sobe, o modelo pode estar a memorizar os exemplos.

5. Aplicabilidade prática e real

O mesmo princípio: observar características e escolher entre duas classes



No exemplo das luas, as características são coordenadas. No mundo real, podem ser valores financeiros, sensores, exames ou padrões de rede.

O exercício das luas é uma simplificação de um problema real: classificação binária. Recebemos várias características e escolhemos entre duas categorias. No mundo real há mais variáveis, mais ruído, custos de erro diferentes e obrigações de explicabilidade.

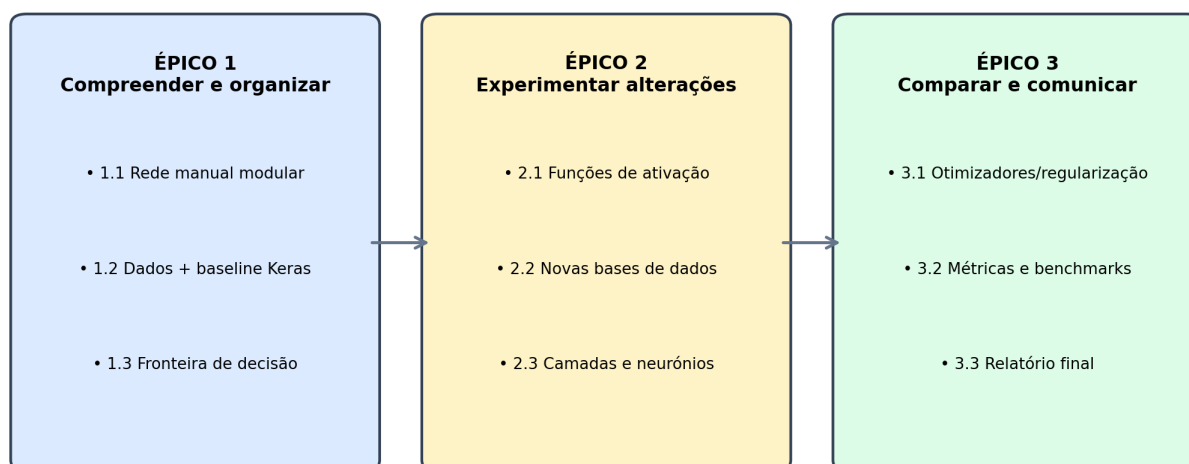
Aplicação	Características	Classes	Cuidado real
Fraude bancária	valor, localização, hora, dispositivo, histórico	transação normal ou suspeita	bloquear uma operação legítima também tem custo
Manutenção preditiva	temperatura, vibração, pressão, corrente	funcionamento saudável ou risco de falha	um falso negativo pode causar avaria
Triagem de saúde	medições clínicas e resultados de exames	baixo risco ou caso a investigar	a rede apoia; não substitui decisão clínica
Cibersegurança	frequência de acesso, origem, padrão de pedidos	atividade legítima ou potencial ataque	os padrões mudam com o tempo
Qualidade industrial	medidas, sensores, imagem da peça	produto aprovado ou defeituoso	é necessário controlar iluminação e calibração

A forma das duas luas não representa literalmente fraude ou doença. Ela demonstra uma ideia: a relação entre características e resposta pode ser curva e complexa, e uma rede neural consegue aprender relações que uma única linha reta não consegue.

6. Plano das tarefas do Jira

O projeto foi dividido em três fases: compreender e organizar; experimentar alterações; comparar e comunicar. Cada tarefa deve terminar com código executável, resultados guardados e uma explicação que outra pessoa consiga reproduzir.

Mapa do projeto NNPROJ



Daniela: reviewer — verifica entendimento, resultados, justificações e qualidade da entrega.

Tarefa	Executor	Pergunta	Pronto quando...
NNPROJ-4 — Rede manual modular	Luciana	Transformar cálculos fixos numa estrutura reutilizável.	A rede aceita dimensões configuráveis; gradientes verificados; comparação com o original.
NNPROJ-5 — Pipeline e baseline Keras	Helton	Separar treino/teste, normalizar e criar referência justa.	Seed fixa; sem fuga de dados; métricas de treino e teste registadas.
NNPROJ-6 — Fronteira de decisão	Pedro	Mostrar visualmente o que a rede aprendeu.	Gráficos antes/depois e legenda clara para o limite 0,5.
NNPROJ-7 — Ativações	Luciana	Comparar sigmoid, tanh, ReLU, LeakyReLU e ELU.	Uma variável alterada de cada vez; tabela e curvas de convergência.

Tarefa	Executor	Pergunta	Pronto quando...
NNPROJ-8 — Novos datasets	Helton	Testar círculos e uma base real.	Pré-processamento adequado; métricas e limitações explicadas.
NNPROJ-9 — Topologias	Pedro	Variar camadas e neurónios.	Número de parâmetros, desempenho e sinais de overfitting comparados.
NNPROJ-10 — Otimizadores/regularização	Luciana	Comparar SGD, momentum, Adam, L2 e dropout.	Mesmos dados/seeds; curvas treino-validação; conclusão justificada.
NNPROJ-11 — Benchmark	Helton	Consolidar accuracy, precision, recall, F1, loss e tempo.	CSV/tabela reproduzível e definição das métricas.
NNPROJ-12 — Relatório	Pedro	Organizar pergunta, método, resultados e conclusão.	Texto revisto por Daniela; figuras numeradas; resultados rastreáveis.

7. Possíveis soluções, código e explicação por tarefa

Os exemplos abaixo são esqueletos pedagógicos. Devem ser executados, completados, comentados e adaptados pela equipa. Os ficheiros correspondentes estão na pasta `guia_rede_neural/solucoes_exemplo`.

7.1 NNPROJ-4 — Modularizar a rede manual

Ideia: Criar objetos para a camada densa e para a ativação. Assim, a dimensão deixa de estar presa a exatamente dois neurónios.

```
"""Possível solução pedagógica para NNPROJ-4.
Generaliza uma camada densa e ativações sem alterar exemplo4.py.
"""

import numpy as np

class DenseManual:
    def __init__(self, entradas, saidas, seed=42):
        rng = np.random.default_rng(seed)
        self.W = rng.normal(0, 0.3, size=(entradas, saidas))
        self.b = np.zeros(saidas)

    def forward(self, X):
        self.X = X
        return X @ self.W + self.b

    def backward(self, grad_saida):
        self.grad_W = self.X.T @ grad_saida / len(self.X)
        self.grad_b = grad_saida.mean(axis=0)
        return grad_saida @ self.W.T

    def atualizar(self, taxa):
        self.W -= taxa * self.grad_W
        self.b -= taxa * self.grad_b

class TanhManual:
    def forward(self, Z):
        self.Y = np.tanh(Z)
        return self.Y

    def backward(self, grad_saida):
        return grad_saida * (1 - self.Y ** 2)
```

Como ler este código: `DenseManual.forward` guarda `X` porque o `backward` precisará dele. O operador `@` é multiplicação matricial. No `backward`, `X.T @ grad_saida` calcula como cada peso influenciou o erro. `atualizar` aplica gradiente descendente.

Verificação sugerida: executar `tarefa_1_1_rede_manual_modular.py`, guardar resultados e comparar com uma baseline usando a mesma `seed` e o mesmo `split`.

7.2 NNPROJ-5 — Pipeline e baseline

Ideia: Separar dados antes de normalizar. O `StandardScaler` aprende apenas no treino e transforma treino/teste com a mesma regra.

```
"""Possível solução pedagógica para NNPROJ-5."""
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from keras import Sequential
from keras.layers import Dense

X, y = make_moons(n_samples=500, noise=0.15, random_state=42)
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
scaler = StandardScaler()
X_treino = scaler.fit_transform(X_treino)
X_teste = scaler.transform(X_teste)

modelo = Sequential([
    Dense(8, input_shape=(2,), activation="tanh"),
    Dense(8, activation="tanh"),
    Dense(1, activation="sigmoid"),
])
modelo.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
historico = modelo.fit(
    X_treino, y_treino, validation_split=0.2,
    epochs=150, batch_size=16, verbose=0
```

```
)
loss, acc = modelo.evaluate(X_teste, y_teste, verbose=0)
print(f"Acurácia de teste: {acc:.3f}")
```

Como ler este código: `stratify` preserva a proporção das classes. `validation_split` acompanha dados não usados nas atualizações. `binary_crossentropy` é a loss habitual para uma saída sigmoid binária.

Verificação sugerida: executar `tarefa_1_2_pipeline_baseline.py`, guardar resultados e comparar com uma baseline usando a mesma seed e o mesmo split.

7.3 NNPROJ-6 — Fronteira de decisão

Ideia: Criar uma grelha de pontos imaginários, pedir uma previsão para cada ponto e pintar o plano segundo a probabilidade.

```
"""Possível solução pedagógica para NNPROJ-6."""
import numpy as np
import matplotlib.pyplot as plt

def plotar_frenteira(modelo, X, y, titulo="Fronteira de decisão"):
    margem = 0.5
    x_min, x_max = X[:, 0].min() - margem, X[:, 0].max() + margem
    y_min, y_max = X[:, 1].min() - margem, X[:, 1].max() + margem
    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 300),
        np.linspace(y_min, y_max, 300),
    )
    pontos = np.c_[xx.ravel(), yy.ravel()]
    probabilidades = modelo.predict(pontos, verbose=0).reshape(xx.shape)
    plt.contourf(xx, yy, probabilidades, levels=20, cmap="RdBu_r", alpha=0.65)
    plt.contour(xx, yy, probabilidades, levels=[0.5], colors="black")
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap="bwr", edgecolor="white")
    plt.title(titulo)
    plt.colorbar(label="Probabilidade da classe 1")
    plt.show()
```

Como ler este código: `meshgrid` constrói a grelha. `contourf` pinta regiões. `contour` desenha a linha em que probabilidade = 0,5, isto é, a fronteira de decisão.

Verificação sugerida: executar `tarefa_1_3_frenteira_decisao.py`, guardar resultados e comparar com uma baseline usando a mesma seed e o mesmo split.

7.4 NNPROJ-7 — Funções de ativação

Ideia: Construir o mesmo modelo várias vezes, alterando somente a ativação.

```
"""Possível solução pedagógica para NNPROJ-7."""
from keras import Sequential
from keras.layers import Dense, Activation, LeakyReLU

def criar_modelo(ativacao="relu"):
    # A camada Dense é igual em todas as experiências.
    # Apenas a função de ativação é alterada.
    modelo = Sequential()
    modelo.add(Dense(16, input_shape=(2,)))

    if ativacao == "leaky_relu":
        modelo.add(LeakyReLU(negative_slope=0.1))
    else:
        modelo.add(Activation(ativacao))

    modelo.add(Dense(1, activation="sigmoid"))
    modelo.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"],
    )
    return modelo

for nome in ["relu", "tanh", "elu", "leaky_relu"]:
    modelo = criar_modelo(nome)
    # Treinar sempre com o mesmo split e seed para uma comparação justa.
    print(nome, modelo.count_params())
```

Como ler este código: A comparação deve manter seed, split, número de camadas, neurónios, épocas e otimizador. Caso contrário, não saberemos qual alteração causou a diferença.

Verificação sugerida: executar tarefa_2_1_ativacoes.py, guardar resultados e comparar com uma baseline usando a mesma seed e o mesmo split.

7.5 NNPROJ-8 — Novas bases

Ideia: Criar funções de preparação com a mesma interface para círculos e para breast cancer.

```
"""Possível solução pedagógica para NNPROJ-8."""
from sklearn.datasets import make_circles, load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

def preparar_circulos():
    X, y = make_circles(n_samples=600, noise=0.08, factor=0.45, random_state=42)
    return separar_normalizar(X, y)

def preparar_cancer():
    dados = load_breast_cancer()
    return separar_normalizar(dados.data, dados.target)

def separar_normalizar(X, y):
    Xtr, Xte, ytr, yte = train_test_split(
        X, y, test_size=0.25, stratify=y, random_state=42
    )
    scaler = StandardScaler()
    return scaler.fit_transform(Xtr), scaler.transform(Xte), ytr, yte
```

Como ler este código: A base real tem muitas características, portanto o input_shape do Keras deve ser X_treino.shape[1], e não 2. A fronteira 2D só é diretamente aplicável a duas características.

Verificação sugerida: executar tarefa_2_2_novos_datasets.py, guardar resultados e comparar com uma baseline usando a mesma seed e o mesmo split.

7.6 NNPROJ-9 — Camadas e neurónios

Ideia: Percorrer combinações de profundidade e largura, treinando e guardando resultados numa tabela.

```
"""Possível solução pedagógica para NNPROJ-9."""
from itertools import product
from keras import Sequential
from keras.layers import Dense

resultados = []
```

```

for numero_camadas, neuronios in product([1, 2, 3, 4], [2, 8, 16, 32, 64]):
    modelo = Sequential()
    modelo.add(Dense(neuronios, input_shape=(2,), activation="tanh"))
    for _ in range(numero_camadas - 1):
        modelo.add(Dense(neuronios, activation="tanh"))
    modelo.add(Dense(1, activation="sigmoid"))
    modelo.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
    # histórico = modelo.fit(...)
    # loss, accuracy = modelo.evaluate(...)
    resultados.append({
        "camadas": numero_camadas,
        "neuronios": neuronios,
        "parametros": modelo.count_params(),
    })

```

Como ler este código: `count_params` mede a complexidade. Mais parâmetros podem melhorar a flexibilidade, mas também aumentar tempo e overfitting. A seleção deve usar validação, não o teste final.

Verificação sugerida: executar `tarefa_2_3_topologias.py`, guardar resultados e comparar com uma baseline usando a mesma seed e o mesmo split.

7.7 NNPROJ-10 — Otimizadores e regularização

Ideia: Criar uma função que devolve variantes controladas do modelo.

```
"""Possível solução pedagógica para NNPROJ-10."""
from keras import Sequential
from keras.layers import Dense, Dropout
from keras.optimizers import SGD, Adam
from keras.regularizers import l2

def criar_modelo(otimizador="adam", usar_regularizacao=True):
    regularizador = l2(1e-4) if usar_regularizacao else None
    modelo = Sequential([
        Dense(32, input_shape=(2,), activation="relu", kernel_regularizer=regularizador),
        Dropout(0.20 if usar_regularizacao else 0.0),
        Dense(16, activation="relu", kernel_regularizer=regularizador),
        Dense(1, activation="sigmoid"),
    ])
    opt = Adam(learning_rate=1e-3) if otimizador == "adam" else SGD(
        learning_rate=0.05, momentum=0.9
    )
    modelo.compile(optimizer=opt, loss="binary_crossentropy", metrics=["accuracy"])
    return modelo
```

Como ler este código: Momentum acumula direção; Adam adapta o tamanho dos passos; L2 penaliza pesos grandes; dropout desliga aleatoriamente unidades durante o treino. Regularização deve ser avaliada pela diferença treino-validação.

Verificação sugerida: executar tarefa_3_1_otimizadores_regularizacao.py, guardar resultados e comparar com uma baseline usando a mesma seed e o mesmo split.

7.8 NNPROJ-11 — Benchmark

Ideia: Executar cada modelo da mesma forma e devolver um dicionário de métricas.

```
"""Possível solução pedagógica para NNPROJ-11."""
from time import perf_counter
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

def avaliar(nome, modelo, X_treino, y_treino, X_teste, y_teste):
    inicio = perf_counter()
    modelo.fit(X_treino, y_treino, epochs=100, batch_size=16, verbose=0)
    segundos = perf_counter() - inicio
    prob = modelo.predict(X_teste, verbose=0).ravel()
    prev = (prob >= 0.5).astype(int)
    return {
        "modelo": nome,
        "accuracy": accuracy_score(y_teste, prev),
        "precision": precision_score(y_teste, prev),
        "recall": recall_score(y_teste, prev),
        "f1": f1_score(y_teste, prev),
        "tempo_s": segundos,
    }

# resultados = [avaliar(...), avaliar(...)]
# pd.DataFrame(resultados).to_csv("benchmark.csv", index=False)
```

Como ler este código: Accuracy mede acertos totais; precision pergunta quantos positivos previstos eram corretos; recall pergunta quantos positivos reais foram encontrados; F1 equilibra precision e recall.

Verificação sugerida: executar tarefa_3_2_benchmark.py, guardar resultados e comparar com uma baseline usando a mesma seed e o mesmo split.

7.9 NNPROJ-12 — Relatório

Ideia: Ler o CSV do benchmark e gerar uma tabela Markdown para reduzir erros de transcrição.

```
"""Possível solução pedagógica para NNPROJ-12.
Gera uma tabela Markdown a partir dos resultados; o texto científico continua a ser escrito pelo grupo.
"""
import pandas as pd

df = pd.read_csv("benchmark.csv")
melhor = df.sort_values("f1", ascending=False).iloc[0]

with open("resumo_resultados.md", "w", encoding="utf-8") as f:
    f.write("# Resumo dos resultados\n\n")
    f.write(df.to_markdown(index=False))
    f.write("\n\n")
```

```
f.write(
    f"O melhor resultado de F1 foi obtido por **{melhor['modelo']}**, "
    f"com F1 = {melhor['f1']:.3f}. Este resultado deve ser interpretado "
    "em conjunto com as curvas de treino e teste.\n"
)
```

Como ler este código: O script automatiza a tabela, mas não substitui a interpretação. O grupo deve explicar por que um modelo funcionou melhor, quais limitações existem e se a diferença é consistente.

Verificação sugerida: executar tarefa_3_3_relatorio.py, guardar resultados e comparar com uma baseline usando a mesma seed e o mesmo split.

8. Como comparar experiências corretamente

Uma experiência científica tenta responder a uma pergunta controlada. Se mudarmos dataset, ativação, arquitetura, épocas e otimizador ao mesmo tempo, não conseguimos atribuir o resultado a uma causa.

- Fixar random_state e, quando possível, seeds do NumPy e TensorFlow.
- Usar o mesmo split treino/validação/teste para modelos comparados.
- Normalizar usando apenas estatísticas do treino.
- Alterar uma variável de cada vez ou declarar claramente um desenho fatorial.
- Executar várias seeds e apresentar média e desvio-padrão, não apenas a melhor execução.
- Não escolher o modelo olhando repetidamente para o teste; usar validação para escolhas e teste uma vez no fim.
- Guardar arquitetura, parâmetros, tempo, curvas e versão das bibliotecas.

Tabela mínima recomendada

Experiência	Dataset	Topologia	Ativação	Otimizador	Val. acc	Teste acc	F1	Tempo
E01	moons	2-5-5-1	relu/tanh	SGD	...	...	...	...
E02	moons	2-8-8-1	tanh	Adam	...	...	...	...

9. Roteiro da reviewer e checklist de entrega

Perguntas que Daniela pode fazer em cada revisão

- Que pergunta esta experiência tenta responder?
- O que foi mantido constante e o que foi alterado?
- O teste usa dados que o modelo nunca viu durante o treino?
- A conclusão é apoiada por números e gráficos?
- O resultado pode ser reproduzido com seed, dependências e instruções?
- O código está explicado pelo autor, ou foi apenas copiado?
- Há sinais de overfitting? Como foram identificados?
- Quais erros seriam mais graves numa aplicação real: falsos positivos ou falsos negativos?

Checklist final

- ☐ Código original preservado e nova versão identificada.
- ☐ README com instalação e execução no VS Code/Jupyter.
- ☐ Treino, validação e teste separados.
- ☐ Seeds e versões registadas.

- ☐ Pelo menos uma alteração de arquitetura/ativação/dataset bem justificada.
- ☐ Gráficos com título, eixos, legenda e fonte dos dados.
- ☐ Tabela comparativa com métricas.
- ☐ Discussão de limitações e aplicações reais.
- ☐ Contribuições dos quatro membros declaradas.
- ☐ Revisão final antes de 26/09/2026.

10. Limitações e melhorias do código atual

Limitação	Por que importa	Melhoria
Sem seed	Resultados mudam entre execuções.	Definir <code>random_state</code> e <code>np.random.seed</code> .
Treino e avaliação nos mesmos 100 pontos	A acurácia pode parecer melhor do que a generalização real.	Criar treino/validação/teste.
Gradientes somados, não promediados	A atualização depende do número de amostras.	Dividir gradientes por <code>len(X)</code> ou usar mini-batches.
Manual e Keras com arquiteturas diferentes	A comparação não é controlada.	Comparar topologias equivalentes primeiro.
MSE na classificação Keras	Pode convergir de forma menos adequada.	Testar <code>binary_crossentropy</code> e justificar.
Resultado Keras não impresso	A avaliação fica invisível.	Imprimir <code>loss</code> e <code>accuracy</code> ou guardar no benchmark.
Poucas métricas	Accuracy pode esconder tipos de erro.	Adicionar matriz de confusão, <code>precision</code> , <code>recall</code> e <code>F1</code> .
Código fixo em dois neurónios	Dificulta experiências de topologia.	Modularizar camadas e dimensões.

Conclusão: o `exemplo4.py` é valioso porque expõe a matemática escondida pelas bibliotecas. O projeto torna-se mais forte quando preserva essa compreensão e, em seguida, introduz práticas reais: dados separados, experiências controladas, métricas adequadas, visualizações e interpretação.

Apêndice — Ficheiros produzidos

- `Guia_Projeto_Rede_Neural.pdf` — este documento.
- `guia_rede_neural/figuras` — gráficos e diagramas gerados.
- `guia_rede_neural/gerar_figuras.py` — script reproduzível das figuras.
- `guia_rede_neural/solucoes_exemplo` — nove soluções pedagógicas separadas.

Os exemplos Keras foram verificados sintaticamente, mas requerem TensorFlow/Keras instalado no ambiente em que forem executados. As imagens empíricas deste guia foram geradas com scikit-learn para permitir validação no ambiente atual.